# adb 工具使用说明

| 版本 | 版本更新说明 | 负责人 | 校审 | 发布日期 |
|---|---|---|---|---|
| V1.0 | 初稿： | 正点原子 linux 团队 | 正点原子 linux 团队 | 2023.06.25 |
| V1.1 | 初稿：基于正点原子 ATK-DLRK3588 adb 工具使用说明，适配 ATK-DLRK3506 | 正点原子 linux 团队 | 正点原子 linux 团队 | 2025.04.16 |



正点原子公司名称 ：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛 ：http://www.openedv.com/forum.php

正点原子淘宝店铺 ：https://openedv.taobao.com

正点原子官方网站 ：www.alientek.com

正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。




扫码关注正点原子公众号

前言 ..... 1
第一章 安装 adb 工具 ..... 2
1.1 adb 简介 ..... 3
1.2 Windows 下安装 adb 工具 ..... 3
1.3 Ubuntu 下安装 adb 工具 ..... 9
第二章 使用 adb 工具 ..... 11
2.1 工作原理介绍 ..... 12
2.2 查看帮助信息 ..... 13
2.3 查看当前连接的设备 ..... 14
2.4 连接设备 ..... 15
2.4.1 连接 RK3XXX Linux buildroot 系统 ..... 15
2.5 运行 shell ..... 18
2.6 文件传输 ..... 19
2.7 应用相关 ..... 20
2.8 启动/停止 adb server ..... 20
2.9 获取 root 权限 ..... 20
2.10 重新挂载 remount ..... 20
2.11 其它命令 ..... 21

# 前言

本文档向用户介绍 adb 工具的安装与使用，包括如下几个章节：

1， 安装 adb 工具，向大家介绍如何在 Windows 系统以及 Ubuntu 系统下安装 adb 工具；

2， 使用 adb 工具，向大家介绍如何使用 adb 工具；

# 第一章 安装 adb 工具

向用户介绍如何在 Windows 系统和 Ubuntu 系统下安装 adb 工具。
本章将分为如下几个小节：
1.1 adb 简介
1.2 Windows 下安装 adb 工具
1.3 Ubuntu 下安装 adb 工具

## 1.1 adb 简介

adb 全称 Android Debug Bridge，直译过来就是 Android 调试桥，它是一个通用的命令行工具。Adb 作为 Android 设备与 PC 端连接的一个桥梁，它可以让开发者通过网络或 USB 与 Android 设备进行通信，从而作为 Android 设备的调试工具。

用户可以通过 adb 在电脑上对 Android 模拟器或者真实的 Android 设备进行一系列操作，比如安装、卸载和调试应用（apk），拷贝推送文件（在电脑与 Android 设备之间拷贝文件），查看设备信息，抓取 log 等操作。它的主要功能包括：

- 运行设备的 shell（命令行）；
- 管理模拟器或设备的端口映射；
- 计算机与设备之间的文件上传/下载；
- 将本地 apk 软件安装至模拟器或 Android 设备。

**需要说明的是，adb 虽然是 Android 设备的调试工具，但随着 adb 的普及，不仅仅是 Android 设备，在嵌入式开发中，很多 Linux 设备也同样支持 adb 调试，例如 Rockchip 平台 Linux 设备。**

## 1.2 Windows 下安装 adb 工具

开发板资料盘中已经给用户提供了 adb 工具的安装包，路径为：**开发板网盘 A 盘-基础资料→05、开发工具→adb 调试工具→platform-tools_r33.0.3-windows.zip**，将 zip 压缩文件解压到 Windows 系统某个目录下，譬如 **D:**，解压得到一个名为“platform-tools”的文件夹，如图 1.2.1 所示：

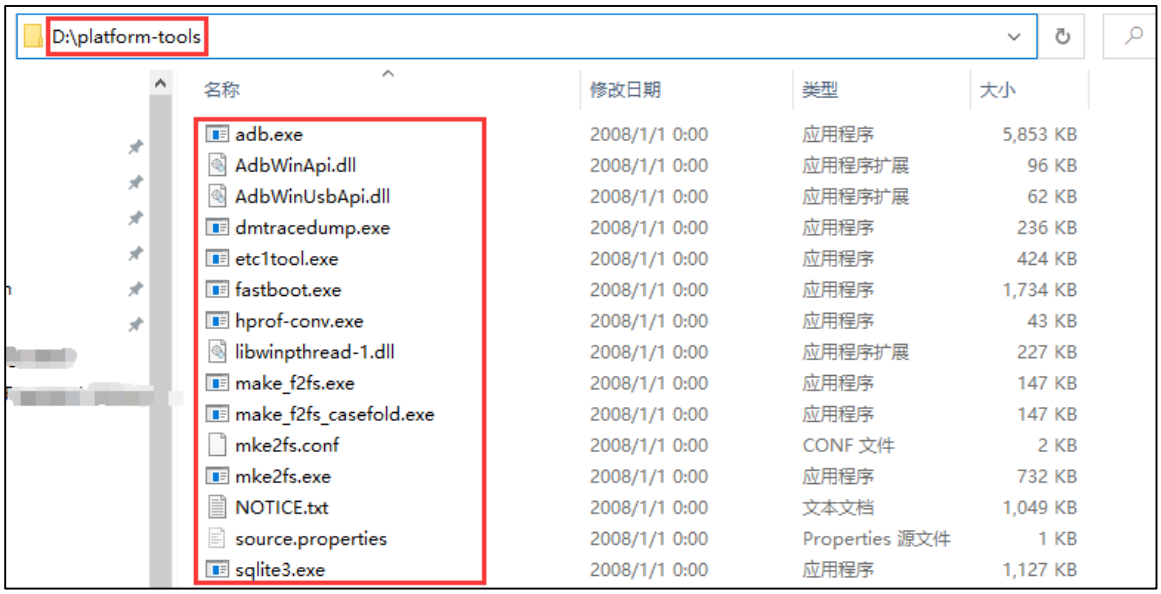



图 1.2.1 解压 platform-tools_r33.0.3-windows.zip

用户也可以通过链接地址 https://developer.android.google.cn/studio/releases/platform-tools，自行下载 adb 安装包，如图 1.2.2 所示：




图 1.2.2 下载 adb 安装包

接下来需要将 platform-tools 目录（也就是 adb 工具的安装目录）添加到 Windows 系统 Path 环境变量中。首先，在 Windows 桌面右键单击“**此电脑**”图标，然后选择“**属性**”，并按照图 1.2.3~1.2.9 所示步骤设置 Path 环境变量：




图 1.2.3 设置 Path 环境变量(1)




图 1.2.4 设置 Path 环境变量(2)

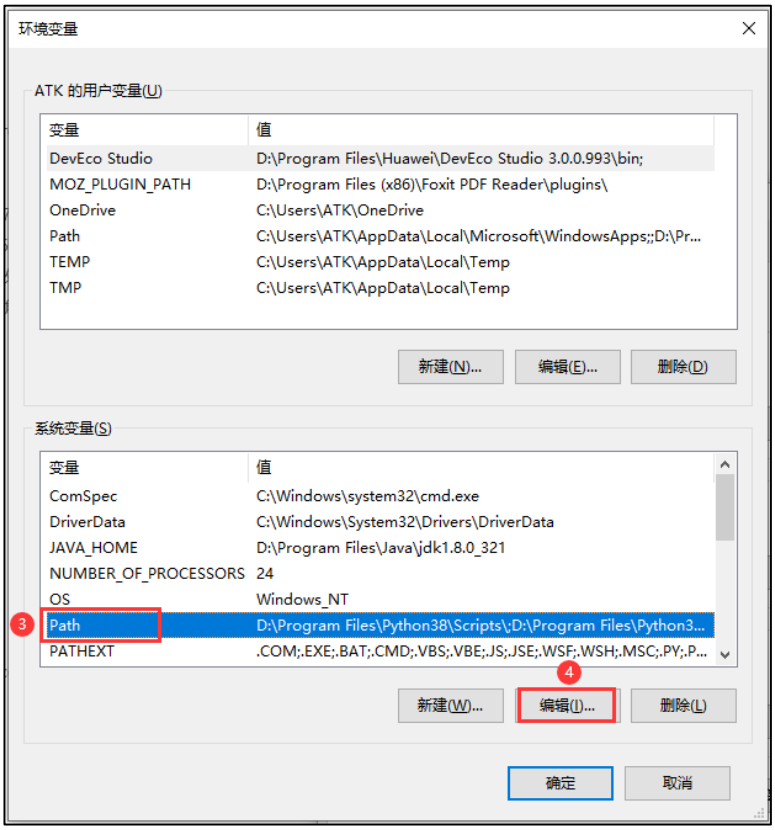


图 1.2.5 设置 Path 环境变量(3)

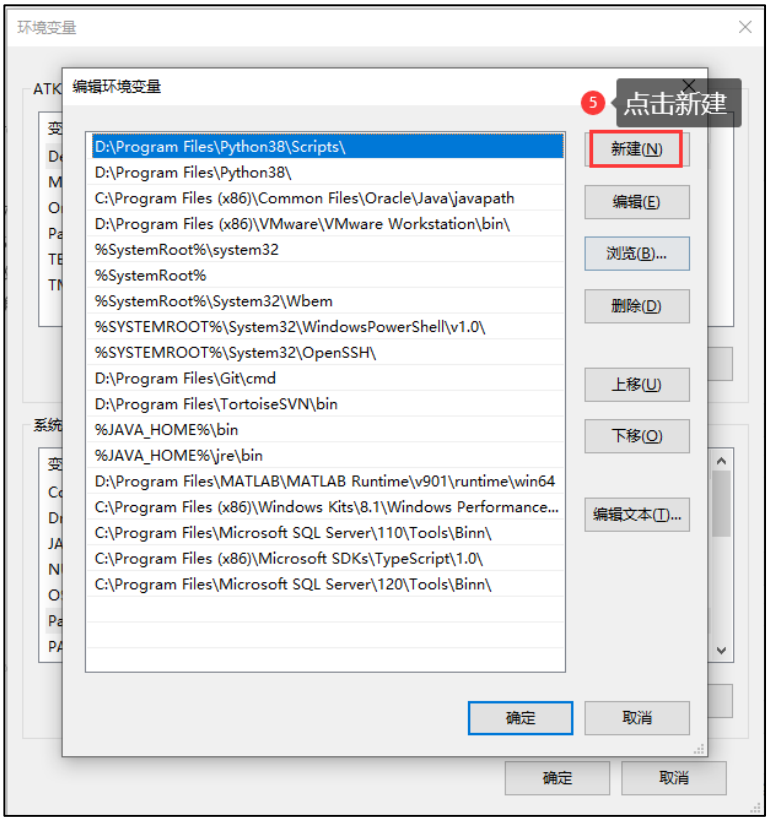



图 1.2.6 设置 Path 环境变量(4)

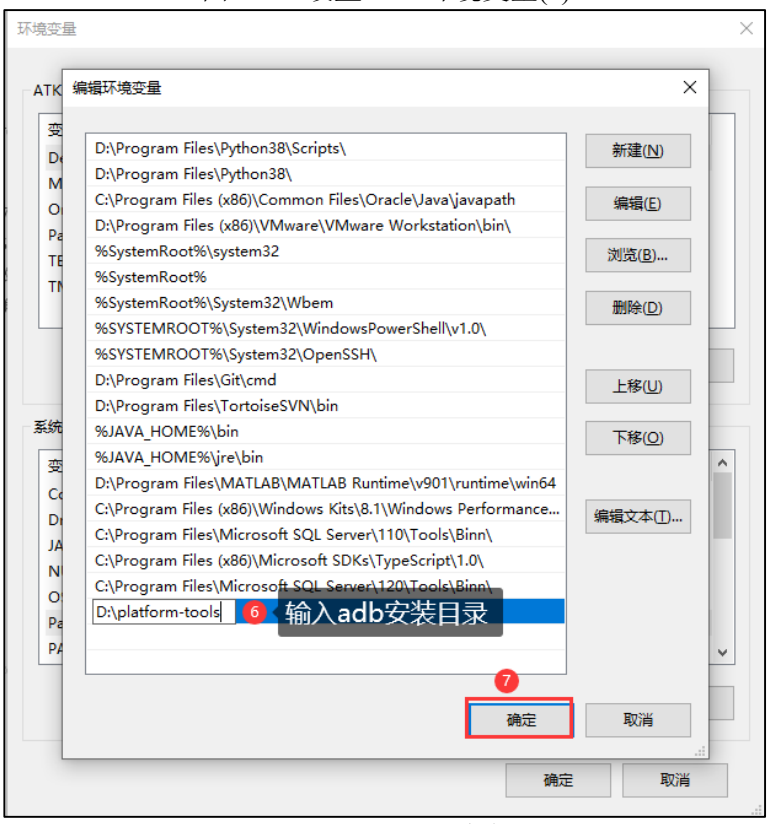



图 1.2.7 设置 Path 环境变量(5)

接下来再点击两次“**确定**”按钮退出窗口：

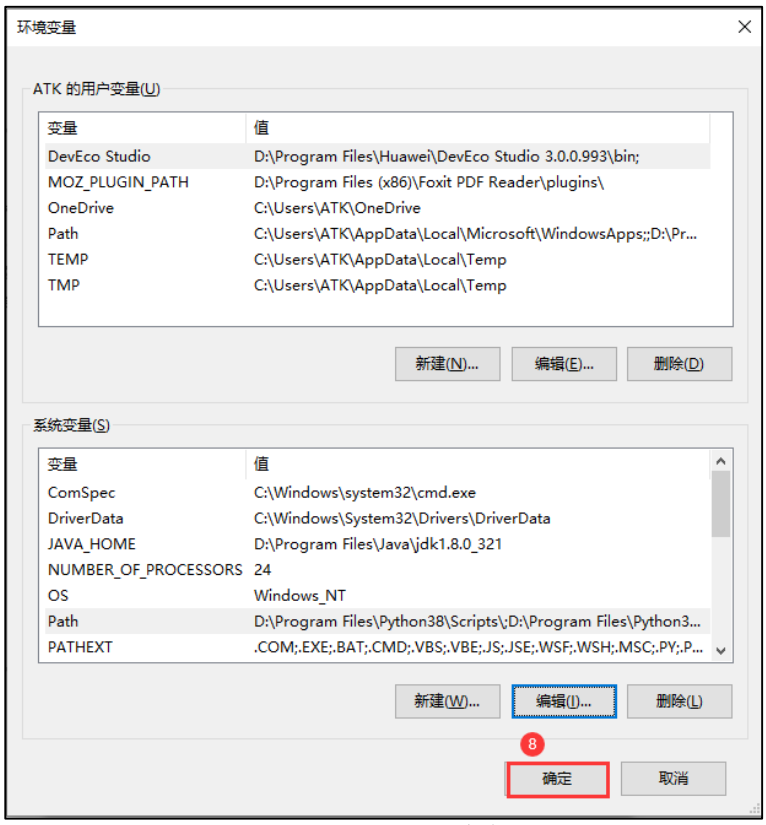



图 1.2.8 设置 Path 环境变量(6)

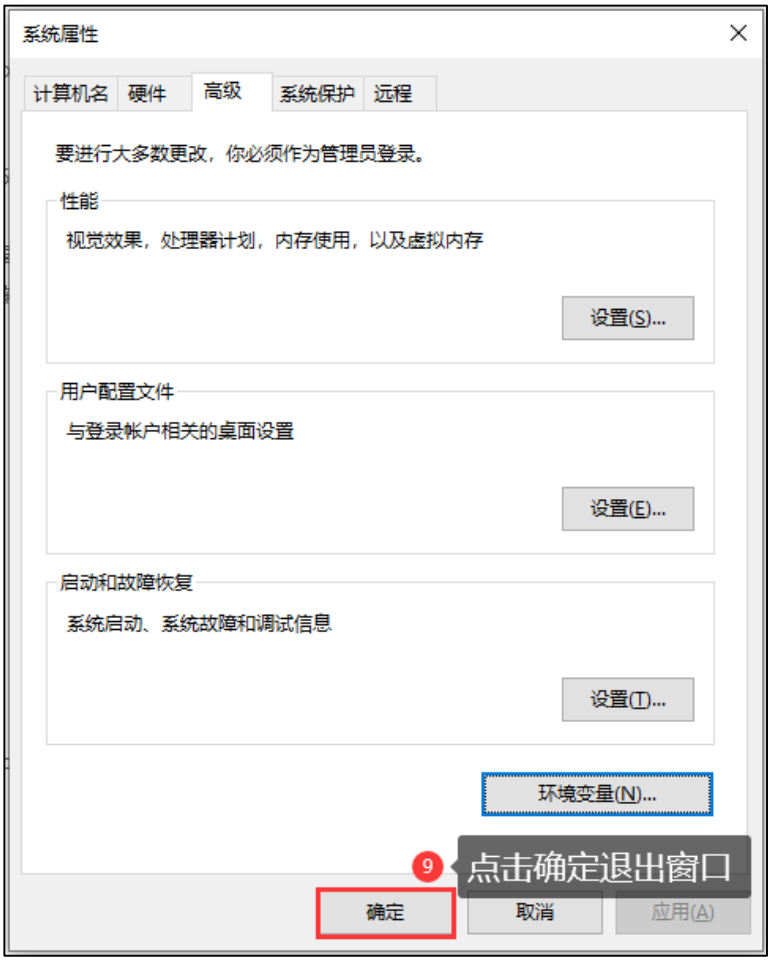


图 1.2.9 设置 Path 环境变量(7)

至此，adb 安装完成。接下来进行测试，按住“**Win+R**”组合键打开 Windows 运行窗口，输入“**cmd**”按回车键打开 Windows 命令行窗口，如图 1.2.10 所示：

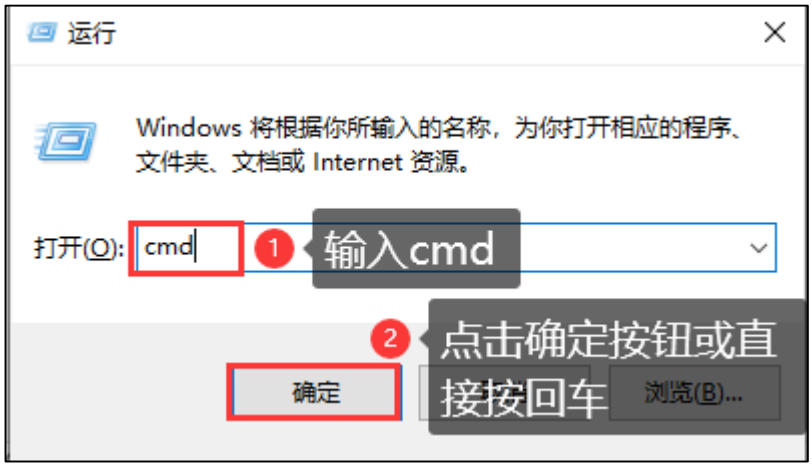



图 1.2.10 Windows 运行窗口




图 1.2.11 Windows cmd 命令行窗口

执行如下命令，确认 adb 安装是否成功：

```
adb version
```

命令执行结果如图 1.2.12 所示：




图 1.2.12 查询 adb 版本

如果能够成功获取到 adb 版本信息，则表示 adb 安装成功！

## 1.3 Ubuntu 下安装 adb 工具

接下来向用户介绍如何在 Ubuntu 系统下安装 adb 工具。开发板资料盘中已经给用户提供了 Linux 系统环境下的 adb 工具安装包，路径为：**开发板网盘 A 盘-基础资料→05、开发工具→adb 调试工具→platform-tools_r33.0.3-linux.zip**，首先打开 Ubuntu 系统的命令行终端，执行如下命令在用户家目录下创建一个名为“**tools**”的目录：

```
mkdir ~/tools
```

将 **platform-tools_r33.0.3-linux.zip** 压缩文件拷贝到 tools 目录下，并执行如下命令将其解压到 tools 目录：

```
cd ~/tools/
unzip platform-tools_r33.0.3-linux.zip
```

```
tgg@tgg-virtual-machine:~$ cd ~/tools/
tgg@tgg-virtual-machine:~/tools$
tgg@tgg-virtual-machine:~/tools$ ls
platform-tools_r33.0.3-linux.zip
tgg@tgg-virtual-machine:~/tools$
tgg@tgg-virtual-machine:~/tools$ unzip platform-tools_r33.0.3-linux.zip
Archive:  platform-tools_r33.0.3-linux.zip
  inflating: platform-tools/NOTICE.txt
  inflating: platform-tools/adb
  inflating: platform-tools/dmtracedump
  inflating: platform-tools/e2fsdroid
  inflating: platform-tools/etc1tool
  inflating: platform-tools/fastboot
  inflating: platform-tools/hprof-conv
  inflating: platform-tools/make_f2fs
  inflating: platform-tools/make_f2fs_casefold
  inflating: platform-tools/mke2fs
  inflating: platform-tools/mke2fs.conf
  inflating: platform-tools/sload_f2fs
 extracting: platform-tools/source.properties
  inflating: platform-tools/sqlite3
  inflating: platform-tools/lib64/libc++.so
tgg@tgg-virtual-machine:~/tools$
tgg@tgg-virtual-machine:~/tools$ ls
platform-tools  platform-tools_r33.0.3-linux.zip
tgg@tgg-virtual-machine:~/tools$
```

图 1.3.1 解压 platform-tools_r33.0.3-linux.zip

解压完成得到 platform-tools 文件夹，其内容如图 1.3.2 所示：

```
tgg@tgg-virtual-machine:~/tools$ cd platform-tools/
tgg@tgg-virtual-machine:~/tools/platform-tools$
tgg@tgg-virtual-machine:~/tools/platform-tools$ ls -l
总用量 17652
-rwxr-xr-x 1 tgg tgg 8104656 1月   1  2008 adb
-rwxr-xr-x 1 tgg tgg   56824 1月   1  2008 dmtracedump
-rwxr-xr-x 1 tgg tgg 1645440 1月   1  2008 e2fsdroid
-rwxr-xr-x 1 tgg tgg  305400 1月   1  2008 etc1tool
-rwxr-xr-x 1 tgg tgg 2417136 1月   1  2008 fastboot
-rwxr-xr-x 1 tgg tgg   12848 1月   1  2008 hprof-conv
drwxrwxr-x 2 tgg tgg    4096 12月  5 15:35 lib64
-rwxr-xr-x 1 tgg tgg  262728 1月   1  2008 make_f2fs
-rwxr-xr-x 1 tgg tgg  262712 1月   1  2008 make_f2fs_casefold
-rwxr-xr-x 1 tgg tgg  879656 1月   1  2008 mke2fs
-rw-r--r-- 1 tgg tgg    1157 1月   1  2008 mke2fs.conf
-rw-r--r-- 1 tgg tgg 1075211 1月   1  2008 NOTICE.txt
-rwxr-xr-x 1 tgg tgg 1701544 1月   1  2008 sload_f2fs
-rw-r--r-- 1 tgg tgg      38 1月   1  2008 source.properties
-rwxr-xr-x 1 tgg tgg 1312888 1月   1  2008 sqlite3
tgg@tgg-virtual-machine:~/tools/platform-tools$
```

图 1.3.2 platform-tools 目录下的文件

执行如下命令将 platform-tools 目录所在路径添加到系统 PATH 环境变量中：

```
export PATH=$PATH:~/tools/platform-tools
```

也可以将该条命令添加至~/.bashrc 文件末尾，这样每次打开新的终端时都会自动执行该条命令：

```
vi ~/.bashrc
```

```
104 if [ -f ~/.bash_aliases ]; then
105     . ~/.bash_aliases
106 fi
107
108 # enable programmable completion features (you don't need to enable
109 # this, if it's already enabled in /etc/bash.bashrc and /etc/profile
110 # sources /etc/bash.bashrc).
111 if ! shopt -oq posix; then
112   if [ -f /usr/share/bash-completion/bash_completion ]; then
113     . /usr/share/bash-completion/bash_completion
114   elif [ -f /etc/bash_completion ]; then
115     . /etc/bash_completion
116   fi
117 fi
118
119 export PATH=$PATH:~/tools/platform-tools 在文件末尾添加这条命令
```

图 1.3.3 .bashrc 文件

添加完成后保存退出！然后执行如下命令使其在当前终端生效：

```
source ~/.bashrc
```

接下来进行测试，执行如下命令，确认 adb 安装是否成功：

```
adb version
```

打印信息如图 1.3.4 所示：

```
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb version
Android Debug Bridge version 1.0.41
Version 33.0.3-8952118
Installed as /home/tgg/tools/platform-tools/adb
tgg@tgg-virtual-machine:~$
```

图 1.3.4 查询 adb 版本

执行命令能成功获取到 adb 版本信息，则表示 adb 安装成功！

# 第二章 使用 adb 工具

本章向用户介绍 adb 工具的使用，包括 adb 的工作原理，以及常用的命令。
本章将分为如下几个小节：

2.1 工作原理介绍
2.2 查看帮助信息
2.3 查看当前连接的设备
2.4 连接设备与断开连接
2.5 运行 shell
2.6 文件传输
2.7 应用相关
2.8 启动/停止 adb server
2.9 获取 root 权限
2.10 重新挂载 remount

本文档仅介绍 adb 工具的常见用法，详细使用方法请参考 Google 官方用户指南：
https://developer.android.google.cn/studio/command-line/adb?hl=zh-cn

## 2.1 工作原理介绍

adb（Android Debug Bridge）它是一种客户端-服务器程序，包括 adb client、adb server 和 adbd 三个部分：

- adb client(客户端)：运行在 PC 上，用于发送命令。用户通过执行 adb 命令来调用客户端。
- adbd(守护进程)：运行在设备上的一个守护进程。adbd 是运行在 Android 设备（模拟器或 Android 真机）或支持 adb 调试的 Linux 设备上的一个守护进程，守护进程在后台运行。当 Android 系统或 Linux 系统启动时，由 init 进程启动 adbd；如果 adbd 进程被终止了，则会由 init 重启。
- adb server(服务器)：运行在 PC 上（作为后台进程运行），作为 adb client 的 server 端、也作为 adbd 守护进程的客户端而存在，用于管理 adb client 和 adbd 之间的通信。

当我们在命令行终端执行 adb 命令时，此时就启动了一个 adb client，该客户端会先检查是否有 adb server 进程正在运行，如果没有，它会启动 adb server 进程。而 adb server 启动后，它会与本地 TCP 端口 5037 进行绑定，并监听 adb client 发出的命令（所有 adb client 均通过端口 5037 与 adb server 进行通信）。

adb server 会与所有正在运行的设备建立连接。首先，它通过扫描 5555 到 5585 之间（该范围供前 16 个模拟器使用）的奇数号端口来查找本地模拟器，adb server 一旦发现 adbd 守护进程，便会与该模拟器建立连接，如图 2.1.1 所示。

请注意，每个模拟器都使用一对按顺序排列的端口 ---- 用于控制台连接的偶数号端口和用于 adb 连接的奇数号端口，例如：

模拟器 1，控制台端口：5554
模拟器 1，adb 端口：5555
模拟器 2，控制台端口：5556
模拟器 2，adb 端口：5557
依此类推…

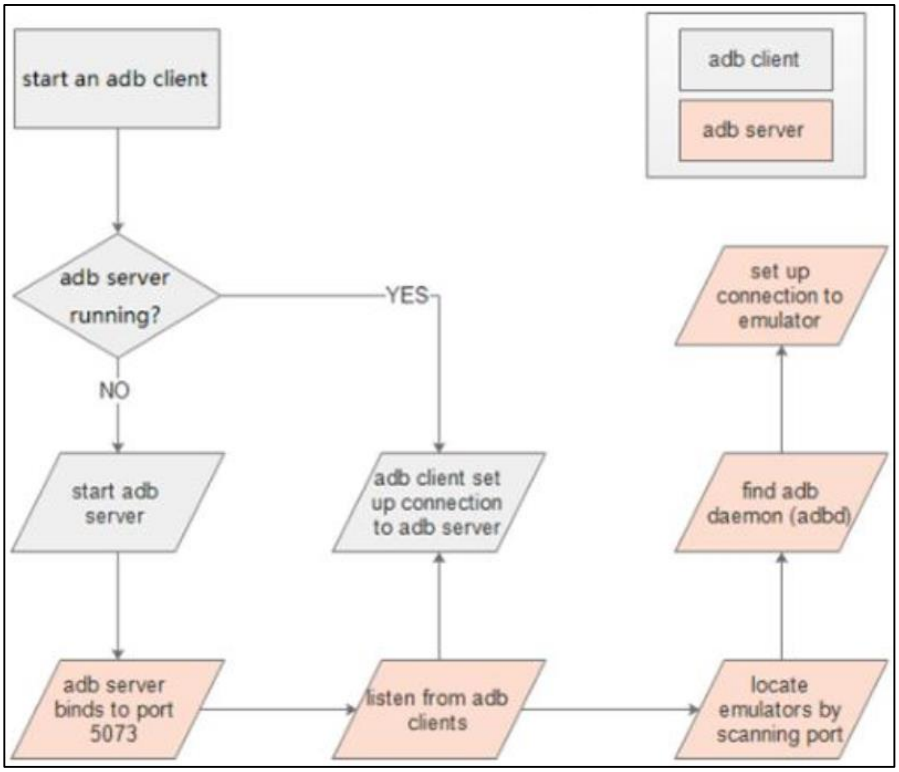



图 2.1.1 adb 工作原理示意图

除了扫描 TCP 端口 5555~5585 来查找本地模拟器之外，adb server 还会扫描 USB 端口来查找所有通过 USB 与 PC 相连的设备，譬如 Android 手机、Android 开发板或支持 adb 调试的 Linux 开发板等，一旦发现 adbd 守护进程，便会与相应的设备建立连接。

adb server 与设备建立连接后，我们便可以在 PC 机上使用 adb 命令访问、操作这些设备。

综上所述：

adb server 会创建 local socket 连接和 remote socket 连接（或 USB 连接）。local socket 用于和本地 adb client 通信，而 remote socket 则用于和设备端 adbd 守护进程通信（如果目标设备通过 USB 连接到 PC，则 adb server 与 adbd 之间通过 USB 通信）。

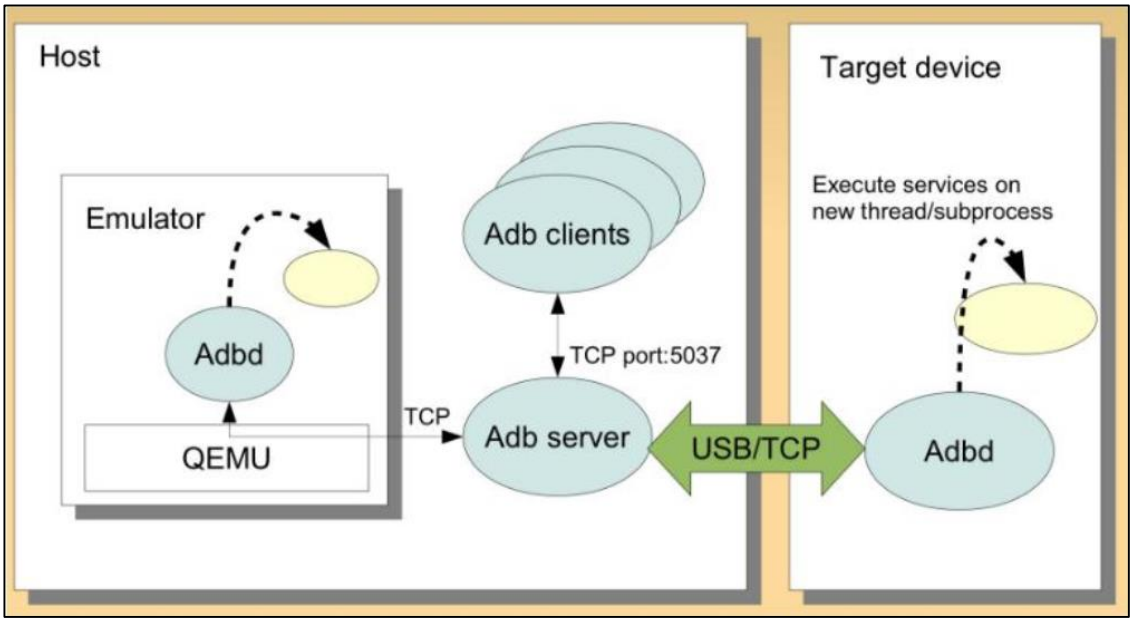



图 2.1.2 adb 工作原理示意图

所以，adb server 是 adb client 的 server 端、同样也是 adbd 的 client 端，它用于管理 adb client 和 adbd 之间的通信，是 adb client 与 adbd 之间的通信中枢（通信中转站）。理解了 adb client、adb server 和 adbd 之间的关系，那么自然就能理解 adb 的工作原理！

从下一小节开始，将向用户介绍如何使用 adb。

## 2.2 查看帮助信息

从本小节开始，将向用户介绍如何使用 adb 工具。adb 是一个通用的命令行工具，可以在 Windows 或者 Linux 系统环境下使用，在第一章内容中，已经向用户介绍了如何在 Windows 以及 Ubuntu 系统下安装 adb 工具。

在 Windows 和 Ubuntu 系统下，adb 的使用方法是相同的，都是通过执行 adb 命令来使用 adb 工具。Windows 下需要打开 cmd 命令行窗口来执行 adb 命令。

文档以 Ubuntu 系统为例，向大家介绍如何使用 adb。首先进入 Ubuntu 系统，打开终端，执行如下命令查看 adb 命令的使用帮助信息：

```
adb help
```

```
tgg@tgg-virtual-machine:~$ adb help
Android Debug Bridge version 1.0.41
Version 33.0.3-8952118
Installed as /home/tgg/tools/platform-tools/adb

global options:
 -a                       listen on all network interfaces, not just localhost
 -d                       use USB device (error if multiple devices connected)
 -e                       use TCP/IP device (error if multiple TCP/IP devices available)
 -s SERIAL                use device with given serial (overrides $ANDROID_SERIAL)
 -t ID                    use device with given transport id
 -H                       name of adb server host [default=localhost]
 -P                       port of adb server [default=5037]
 -L SOCKET                listen on given socket for adb server [default=tcp:localhost:5037]
 --one-device SERIAL|USB  only allowed with 'start-server' or 'server nodaemon', server will only connect to one USB
 --exit-on-write-error    exit if stdout is closed

general commands:
 devices [-l]             list connected devices (-l for long output)
 help                     show this help message
 version                  show version num

networking:
 connect HOST[:PORT]      connect to a device via TCP/IP [default port=5555]
 disconnect [HOST[:PORT]]
     disconnect from given TCP/IP device [default port=5555], or all
 pair HOST[:PORT] [PAIRING CODE]
     pair with a device for secure TCP/IP communication
 forward --list           list all forward socket connections
 forward [--no-rebind] LOCAL REMOTE
     forward socket connection using:
       tcp:<port> (<local> may be "tcp:0" to pick any open port)
       localabstract:<unix domain socket name>
       localreserved:<unix domain socket name>
       localfilesystem:<unix domain socket name>
       dev:<character device name>
       jdwp:<process pid> (remote only)
       vsock:<CID>:<port> (remote only)
       acceptfd:<fd> (listen only)
 forward --remove LOCAL   remove specific forward socket connection
 forward --remove-all     remove all forward socket connections
 reverse --list           list all reverse socket connections from device
 reverse [--no-rebind] REMOTE LOCAL
     reverse socket connection using:
       tcp:<port> (<remote> may be "tcp:0" to pick any open port)
```

图 2.2.1 查看 adb 帮助信息

adb 命令支持多种参数搭配来实现不同的操作。

## 2.3 查看当前连接的设备

执行如下命令可以查看当前连接的设备：

```
adb devices
```




图 2.3.1 查询当前连接的设备

当我们执行 adb 命令时，此时就启动了一个 adb client，adb client 会先检查是否有 adb server 进程正在运行，如果没有，它会先启动 adb server 进程，并输出如下信息：

```
* daemon not running; starting now at tcp:5037
* daemon started successfully
```

当输出“**daemon started successfully**”信息时，表示 adb server 已经启动成功了。

adb server 启动成功后，它会与本地 TCP 端口 5037 进行绑定，并监听 adb client 发出的命令。“**adb devices**”命令用于查询当前连接的设备，并将其显示在“**List of devices attached**”下方；当前我这里并未连接任何设备，所以显示为空。

## 2.4 连接设备

本小节介绍如何连接设备，包括 ATK-DLRK3588 开发板运行不同的系统时如何去连接，譬如 Android 系统、Linux buildroot 系统。

### 2.4.1 连接 RK3XXX Linux buildroot 系统

可以通过网络（有线网络、无线网络 WiFi）或 USB 连接设备。

#### 1. 通过 USB 连接

以 ATK-DLRK3588 为例:

首先 ATK-DLRK3588 开发板烧录 Linux buildroot 系统镜像，然后使用 USB 线一头连接开发板的 TYPE-C0 口(ATK-DLRK3506B 请连接板上的 USB 字样的 type-c 接口)、另一头连接到电脑的 USB 口，如图 2.4.1.1 所示：

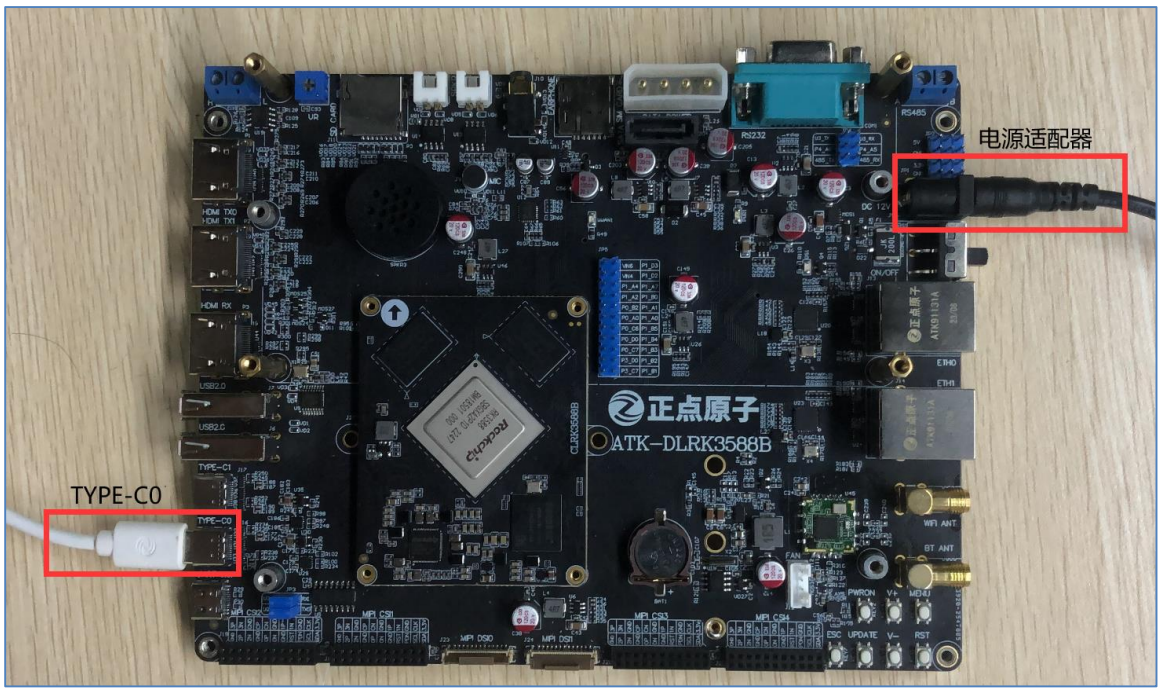



图 2.4.1.1 开发板硬件连接

连接好硬件上电启动开发板，进入 Linux buildroot 系统后，打开 Ubuntu 系统，在“**虚拟机→可移动设备**”下查找是否存在一个名为“**Fuzhou Rockchip Android ADB Interface**”的设备，如下图所示：

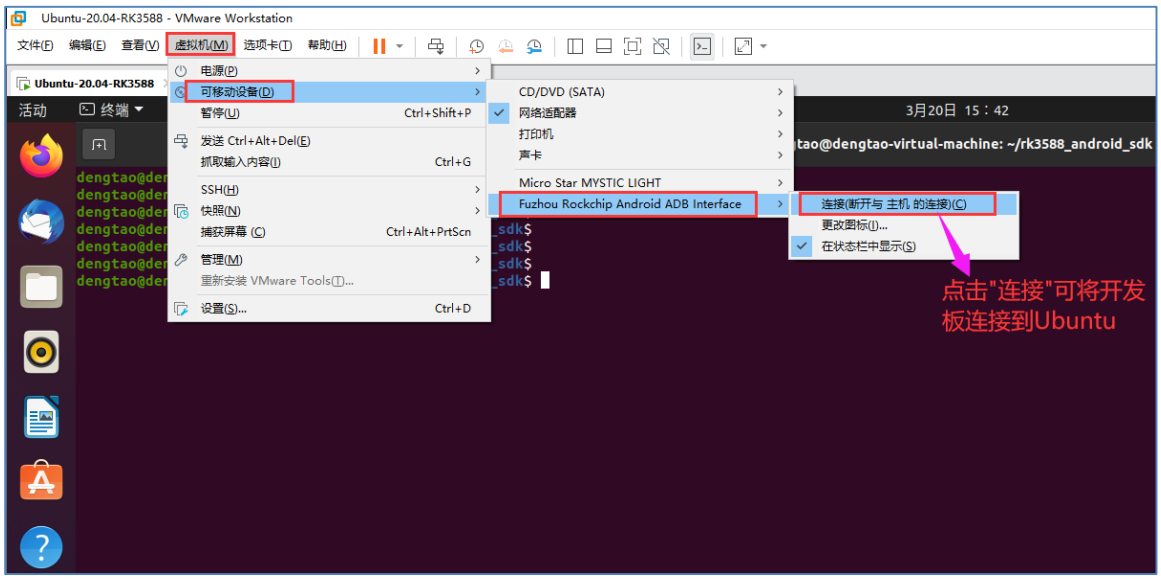



图 2.4.1.2 开发板连接到 Ubuntu

如果存在该设备则表示开发板已经连接到 PC 机了；如果不存在该设备，请检查硬件连接是否有误，并尝试重新拔插 USB 线或更换电脑的 USB 口。鼠标点击“**连接(断开与 主机 的连接)(C)**”将开发板连接到 Ubuntu。

连接成功后，Ubuntu 左侧导航栏会出现一个类似手机的图标（稍微等待一会），如图 2.4.1.3 所示：

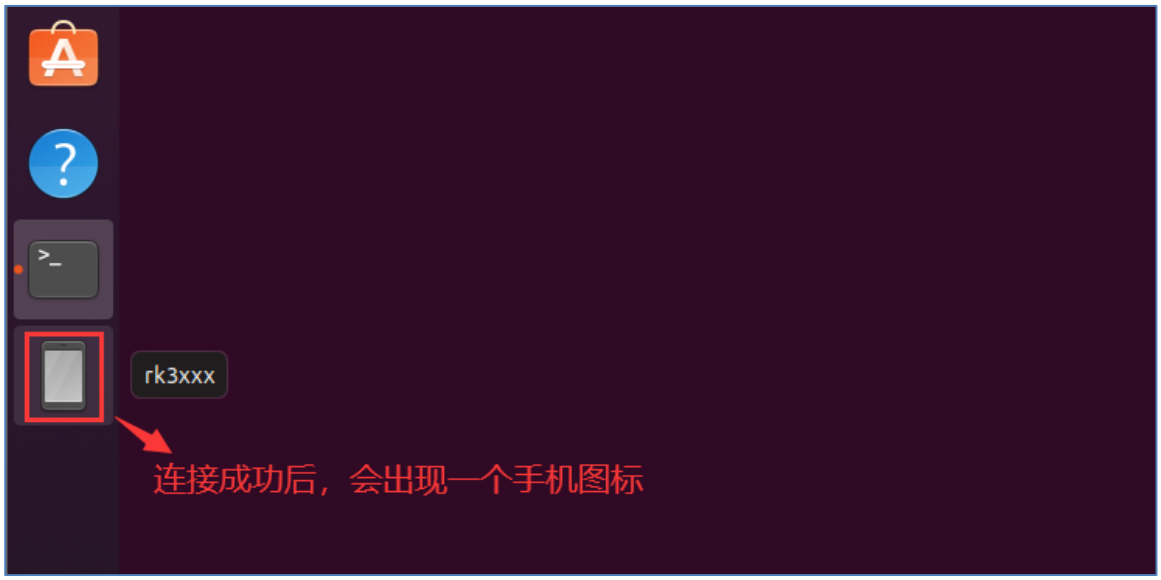



图 2.4.1.3 连接成功

此时在终端执行“**adb devices**”命令：

```
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb devices
List of devices attached
b9f7f8a975b888fe        device

tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$
```

图 2.4.1.4 查询当前连接的设备

从上图可知，当前已经有一个设备和 Ubuntu 建立了 adb 连接，“**b9f7f8a975b888fe**”是设备的序列号。

**2. 通过网络连接**

以 ATK-DLRK3588 为例：

ATK-DLRK3588 开发板运行 Linux buildroot 系统，准备一根网线，将开发板的网口（两个千兆网口中的任何一个均可）（ATK-DLRK3506B 底板上的是两个百兆网络）连接到路由器（或者直接使用 WiFi 连接无线网络），如下图所示：

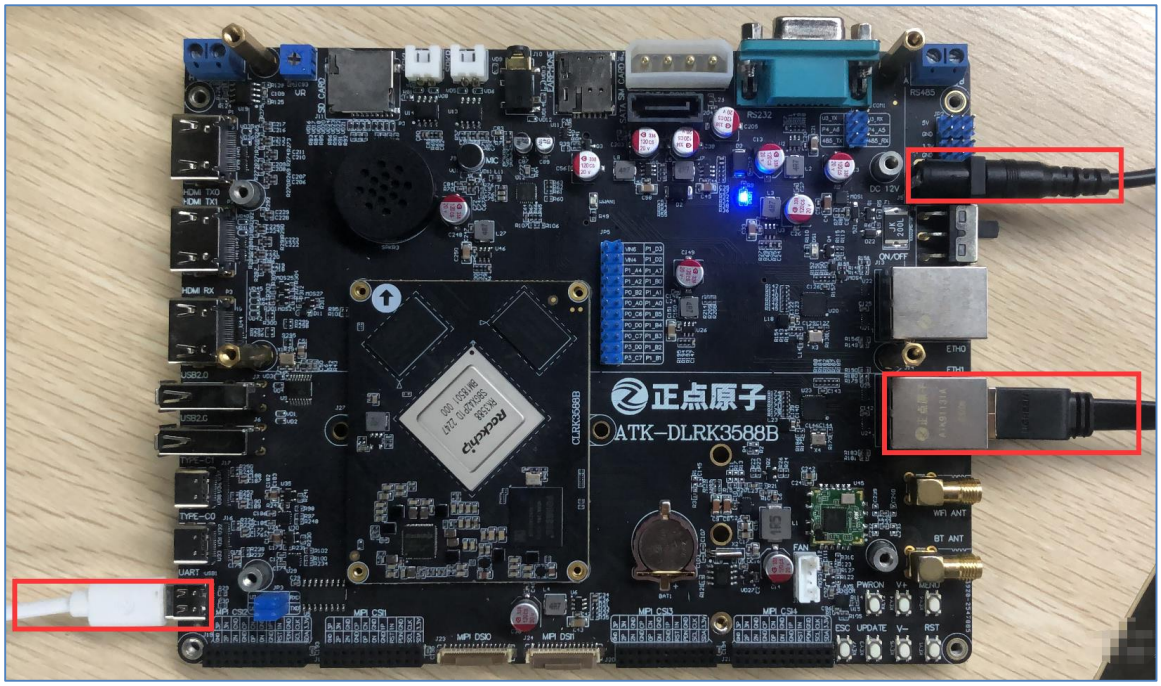



图 2.4.1.5 开发板硬件连接

在串口终端执行“**ifconfig**”命令查看开发板的 IP 地址：

```
root@RK356X:/# ifconfig
eth1      Link encap:Ethernet  HWaddr E2:C5:89:F2:98:07
          inet addr:192.168.6.159  Bcast:192.168.6.255  Mask:255.255.255.0
          inet6 addr: fe80::cbad:c741:d826:bad0/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:536 errors:0 dropped:0 overruns:0 frame:0
          TX packets:97 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:54483 (53.2 KiB)  TX bytes:8375 (8.1 KiB)
          Interrupt:50

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:68 errors:0 dropped:0 overruns:0 frame:0
          TX packets:68 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4976 (4.8 KiB)  TX bytes:4976 (4.8 KiB)

p2p0      Link encap:Ethernet  HWaddr 8E:06:DD:E7:8E:9F
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

usb0      Link encap:Ethernet  HWaddr 6A:94:5D:3F:41:16
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

wlan0     Link encap:Ethernet  HWaddr 8C:06:DD:E7:8E:9F
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

图 2.4.1.6 查询 IP 地址

确保电脑与开发板连接在同一个局域网内，此时可以通过如下命令连接开发板：

```
adb connect ip:port
```

ip 表示 RK3588 开发板的 IP 地址，port 表示端口号，端口号有默认值（默认端口号为 5555），可以不用填，如下所示：

```
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb connect 192.168.6.159
connected to 192.168.6.159:5555 表示连接设备成功
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb devices
List of devices attached
192.168.6.159:5555      device

tgg@tgg-virtual-machine:~$
```

图 2.4.1.7 连接设备

连接成功后，通过“**adb devices**”命令查看已连接的设备。

使用“**adb disconnect**”命令可断开与该设备的连接：

```
adb disconnect ip:port
```

譬如：

```
adb disconnect 192.168.6.159:5555
```

## 2.5 运行 shell

执行“**adb shell**”命令可以运行设备端的 shell（命令行终端），如下所示：

```
adb shell
```




图 2.5.1 运行设备的 shell

可执行 exit 退出 shell。

如果连接了多台设备，则可以使用-s 选项来指定哪一台设备，如下所示：




图 2.5.2 指定设备

也可以执行如下命令、在设备端运行一条 shell 命令：

```
adb shell command
```

command 表示需要在设备端运行的 Linux 命令，譬如：

```
adb shell ls
adb shell ifconfig
adb shell pwd
adb shell echo "Hello world"
```

## 2.6 文件传输

通过“adb push”命令可以将本地文件拷贝至设备：

```
adb push local remote
```

local 表示本地文件，remote 表示远端设备的路径，譬如将本地文件 helloWorld 拷贝至设备的/sdcard/目录：

```
adb push helloWorld /sdcard
```

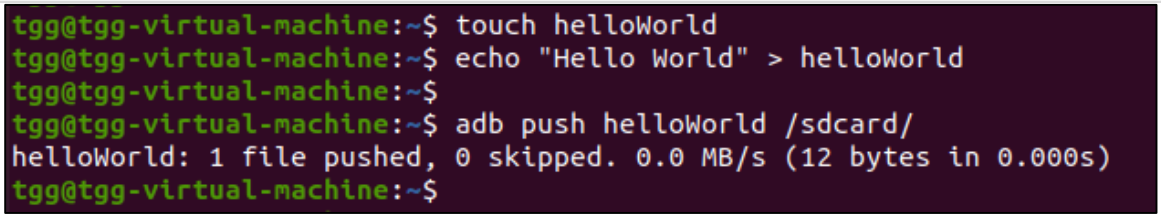


图 2.6.1 文件拷贝

此时设备的/sdcard/目录下便会存在 helloWorld 文件：

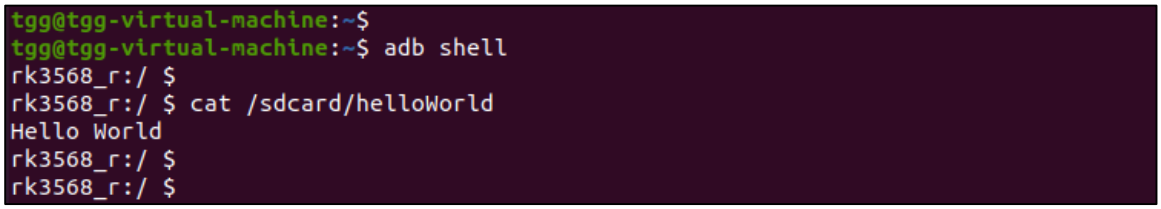


图 2.6.2 查看文件内容

通过“**adb pull**”命令可以将远端设备的文件拷贝至本地：

```
adb pull remote local
```

remote 表示远端设备的文件，local 表示本地路径，譬如将设备的/sdcard/helloWorld 文件拷贝至本地：

```
tgg@tgg-virtual-machine:~$ rm -rf helloWorld
tgg@tgg-virtual-machine:~$ adb pull /sdcard/helloWorld ./
/sdcard/helloWorld: 1 file pulled, 0 skipped. 0.0 MB/s (12 bytes in 0.047s)
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ cat helloWorld
Hello World
tgg@tgg-virtual-machine:~$
```

图 2.6.3 文件拷贝

## 2.7 应用相关

执行如下命令可以在 Android 模拟器或 Android 设备上安装 APK：

```
adb install path_to_apk
```

path_to_apk 表示 APK 的路径。

## 2.8 启动/停止 adb server

有时我们需要重启 adb server，可以先执行“adb kill-server”命令将 adb server 终止，然后再执行“adb start-server”命令启动 adb server，实现重启操作：

```
adb kill-server
adb start-server
```

```
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb kill-server
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb start-server
* daemon not running; starting now at tcp:5037
* daemon started successfully
tgg@tgg-virtual-machine:~$
```

图 2.8.1 重启 adb server

## 2.9 获取 root 权限

通过“adb root”命令获取设备的 root 权限，使得 adb 操作具有更高的权限。

```
adb root
```

## 2.10 重新挂载 remount

我们在使用“**adb push**”命令拷贝文件时，可能会失败，并且会有如下提示信息：“**Read-only file system**”：

```
remote secure_mkdirs failed: Read-only file system
```

这个信息表示设备端的文件系统是只读模式，无法写入。这个时候需要进行重新挂载，将文件系统重新挂载成可读可写模式，执行如下命令：

```
adb root
adb remount
```

```
tgg@tgg-virtual-machine:~$ adb remount
Using overlayfs for /system
Using overlayfs for /vendor
Using overlayfs for /odm
Using overlayfs for /product
Using overlayfs for /system_ext
Now reboot your device for settings to take effect
remount succeeded
tgg@tgg-virtual-machine:~$
```

执行“**adb remount**”命令之前，需要先执行“**adb root**”命令获取到设备的 root 权限。重新挂载之后，再次 push 时就不会出现问题了。

## 2.11 其它命令

更多使用方法请参考：https://developer.android.google.cn/studio/command-line/adb?hl=zh-cn